Course: CS420 – Artificial Intelligence

Midterm examination review – Lectures 0 to 3

PRACTICE MIDTERM EXAMINATION – TEST 3
SOLUTION

Duration: 120 minutes · Closed book · Calculator allowed

Unless a question says otherwise, ties are broken in alphabetical order (left-to-right in trees), and successors are generated in alphabetical order.

Question 1. *Tower of Hanoi with 3 pegs and 3 disks.* Three disks of different sizes are stacked on peg 1 (largest at the bottom). The goal is to move the stack to peg 3, moving one disk at a time and never placing a larger disk on a smaller one.

(a) Consider a state as a tuple of values. Which values should be included in the tuple?

> (p_1, p_2, p_3) = the peg of the small, medium and large disk. On a peg the order of the disks is forced by their sizes, so it does not need to be stored.

(b) Define the initial state and the goal state.

> Initial $(1, 1, 1)$; goal $(3, 3, 3)$.

(c) Define the action function.

> Move the top disk of peg a (the smallest i with $p_i = a$) to peg $b \neq a$, provided peg b is empty or its top disk is larger; the result sets $p_i = b$.

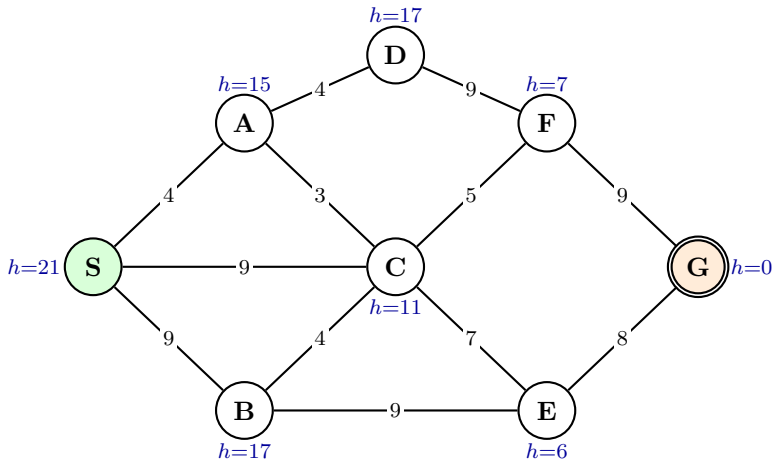
(d) What is the action cost?

> 1 per move.

(e) What is the maximum number of states? How many are reachable? What is the length of an optimal solution?

> $3^3 = \mathbf{27}$ tuples; every tuple is legal and all **27** are reachable. Optimal solution: $2^3 - 1 = \mathbf{7}$ moves.

Question 2. Consider the undirected graph below; S is the initial state and G the goal state.



For each search strategy, (a) list, in order, the states **expanded**, (b) give the path found and (c) give the final content of the frontier (a state is expanded when it is removed from the frontier). Conventions:

BFS, DFS and greedy best-first search test a state for the goal when it is *generated* (so the goal is not expanded); UCS and A* test it when it is *removed* from the frontier (graph search; a cheaper path to a frontier state replaces its entry). DFS avoids loops by remembering the states on the current path only. The path returned will not be accepted if the list of expanded states is wrong.

Algorithm	**List of expanded states**	**Path returned**	**Final frontier**
Breadth-first search	S A B C D E	S B E G (cost 26)	{ F }
Uniform-cost search	S A C D B F E G	S A C F G (cost 21)	{ }
Depth-first search	S A C B E	S A C B E G (cost 28)	(not asked)
Greedy best-first search	S C E	S C E G (cost 24)	{ F, A, B }
Graph-search A*	S A C F E G	S A C F G (cost 21)	{ D, B }

BFS (queue after each expansion):

Step	Expanded	Queue (front → back) after the expansion
1	S	A, B, C
2	A	B, C, D
3	B	C, D, E
4	C	D, E, F
5	D	E, F
6	E	generates goal **G** ⇒ stop; frontier: F

DFS (stack after each expansion):

Step	Expanded	Stack (top → bottom) after the expansion
1	S	A, B, C
2	A	C, D, B, C
3	C	B, E, F, D, B, C
4	B	E, E, F, D, B, C
5	E	generates goal **G** ⇒ stop; frontier: E, F, D, B, C

UCS:

Step	Expanded	Frontier after the expansion, in priority order (subscript = $g(n)$)
1	S_0	A_4, B_9, C_9
2	A_4	C_7, D_8, B_9
3	C_7	D_8, B_9, F_{12}, E_{14}
4	D_8	B_9, F_{12}, E_{14}
5	B_9	F_{12}, E_{14}
6	F_{12}	E_{14}, G_{21}
7	E_{14}	G_{21}
8	G_{21}	*goal removed from the frontier – stop*

Greedy best-first:

Step	Expanded	Frontier after the expansion, in priority order (subscript = $h(n)$)
1	S_{21}	C_{11}, A_{15}, B_{17}
2	C_{11}	E_{6}, F_{7}, A_{15}, B_{17}
3	E_{6}	generates goal **G** ⇒ stop; frontier: F_{7}, A_{15}, B_{17}

A*:

Step	Expanded	Frontier after the expansion, in priority order (subscript = $g + h = f$)
1	$S_{0+21=21}$	$A_{4+15=19}$, $C_{9+11=20}$, $B_{9+17=26}$
2	$A_{4+15=19}$	$C_{7+11=18}$, $D_{8+17=25}$, $B_{9+17=26}$
3	$C_{7+11=18}$	$F_{12+7=19}$, $E_{14+6=20}$, $D_{8+17=25}$, $B_{9+17=26}$
4	$F_{12+7=19}$	$E_{14+6=20}$, $G_{21+0=21}$, $D_{8+17=25}$, $B_{9+17=26}$
5	$E_{14+6=20}$	$G_{21+0=21}$, $D_{8+17=25}$, $B_{9+17=26}$
6	$G_{21+0=21}$	*goal removed from the frontier – stop*

(d) Is h admissible? Justify.

h^*: S= 21, A= 17, B= 17, C= 14, D= 18, E= 8, F= 9, G= 0; $h \leq h^*$ everywhere ⇒ **admissible**.

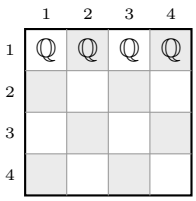
(e) Is h consistent? List every violated inequality.

No: $h(S) = 21 > c(S,A) + h(A) = 19$; $h(S) = 21 > c(S,C) + h(C) = 20$; $h(A) = 15 > c(A,C) + h(C) = 14$; $h(B) = 17 > c(B,C) + h(C) = 15$; $h(B) = 17 > c(B,E) + h(E) = 15$; $h(D) = 17 > c(D,F) + h(F) = 16$.

(f) Decrease as few h-values as possible so that h becomes consistent. Does the new heuristic dominate the old one, or the opposite?

Lower $h(S)$ from 21 to 18, $h(A)$ from 15 to 14, $h(B)$ from 17 to 15, $h(D)$ from 17 to 16. Every value was only decreased, so the new h' is still admissible, and the *old* h dominates h' ($h \geq h'$ everywhere). Dominance alone is not enough here: the lecture's guarantee that the dominating heuristic expands no more nodes assumes both are consistent.

Question 3. The 4-queens problem asks to place four queens on a 4×4 board so that no queen attacks another. States are configurations with all four queens on the board, one per column; a successor moves one queen to another square of its column; h(state) is the number of attacking pairs. The initial state S (written as rows $1, 1, 1, 1$; row 1 at the top) is shown below.



(a) How many states are there in total? How many successors does each state have? Explain.

$4^4 =$ **256** states (each queen in any row of its column); $4 \times 3 =$ **12** successors.

(b) What is $h(S)$? Explain.

All four queens share row 1: every pair attacks, $h(S) = \binom{4}{2} =$ **6**.

(c) Apply steepest-descent hill climbing from S. At every step write the h value of each of the 12 successors on the board (in the square the queen would move to) and choose the best one (ties: left-most column, then top row). Stop at a solution or when no successor is better.

Numbers = h after moving the queen of that column to that square; red = chosen move.

	1	2	3	4
1	♕	♕	♕	♕
2	4	5	5	4
3	4	4	4	4
4	4	**3**	3	4

state $(1,1,1,1)$, $h=6$

	1	2	3	4
1	♕	6	♕	♕
2	**1**	5	2	3
3	3	4	3	1
4	3	♕	2	3

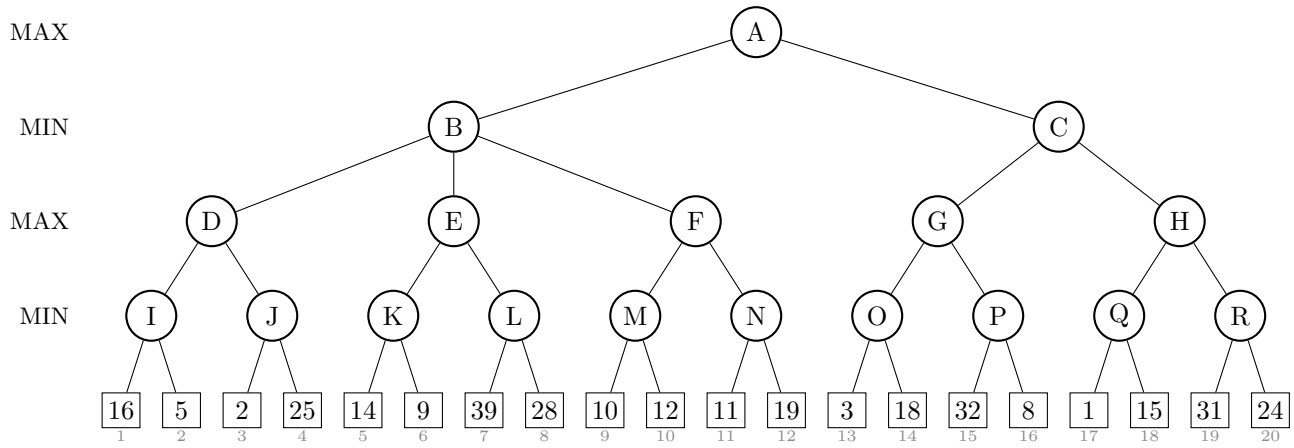
state $(1,4,1,1)$, $h=3$

	1	2	3	4
1	3	4	♕	♕
2	♕	3	2	3
3	3	3	1	**0**
4	3	♕	2	1

state $(2,4,1,1)$, $h=1$

Level 3: state $(2,4,1,3)$ with $h=0$ — **solved**. Hill climbing reaches a solution in 3 moves here (from other states, e.g. $(4,3,2,1)$, it gets stuck at $h=1$ without sideways moves).

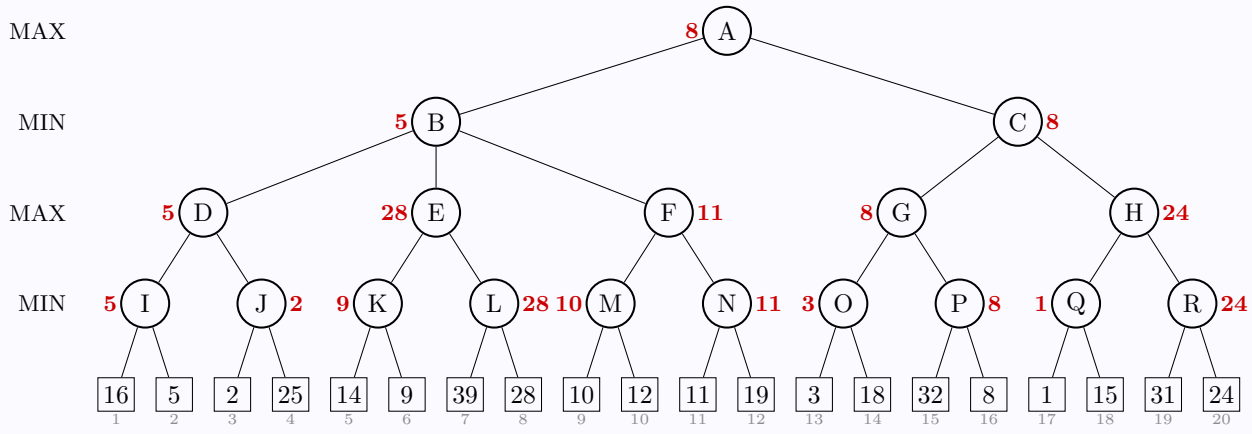
Question 4. Consider the following 4-ply game tree (MAX at the root, children visited left to right).



(a) Compute the final backed-up value of every node using the minimax algorithm (no alpha-beta pruning at this step). Which move should MAX make at the root?

Node	**A**	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**
Minimax	8	5	8	5	28	11	8	24	5

Node	**J**	**K**	**L**	**M**	**N**	**O**	**P**	**Q**	**R**
Minimax	2	9	28	10	11	3	8	1	24



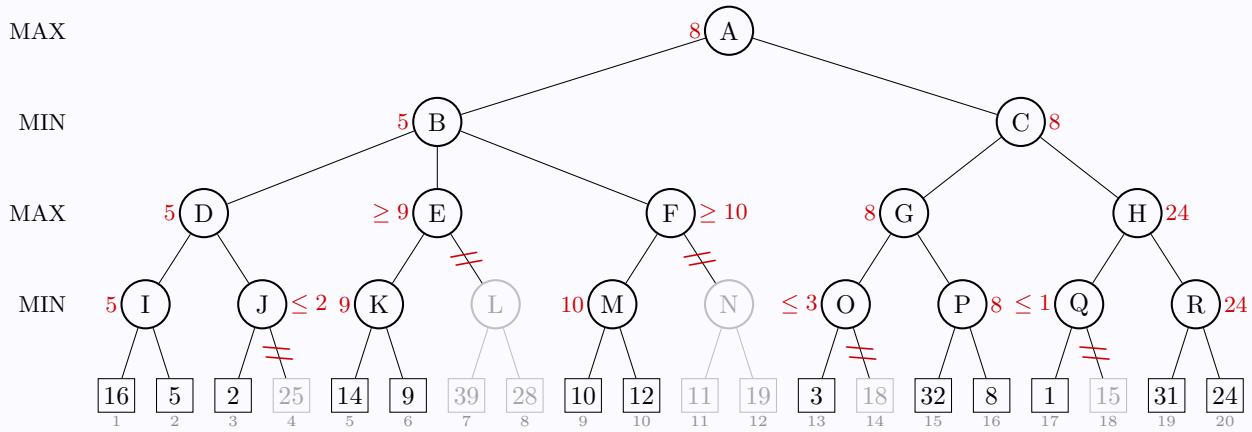
Root value = 8; MAX moves to **C**.

(b) Run alpha-beta pruning (children visited left to right). Give the value computed at every node (mark **X** if the node is never visited), cross out the pruned branches on the tree and briefly explain each case.

Node	**A**	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**
α-β	8	5	8	5	≥ 9	≥ 10	8	24	5

Node	**J**	**K**	**L**	**M**	**N**	**O**	**P**	**Q**	**R**
α-β	≤ 2	9	**X**	10	**X**	≤ 3	8	≤ 1	24

($\leq v$ / $\geq v$: the node was cut off, so only this bound is known; writing the returned number v is also accepted.)



α and β at every visited node:

Node	Type	On entry	When it returns	Value returned
A	MAX	$\alpha = -\infty,\ \beta = +\infty$	$\alpha = 8,\ \beta = +\infty$	8
B	MIN	$\alpha = -\infty,\ \beta = +\infty$	$\alpha = -\infty,\ \beta = 5$	5
D	MAX	$\alpha = -\infty,\ \beta = +\infty$	$\alpha = 5,\ \beta = +\infty$	5
I	MIN	$\alpha = -\infty,\ \beta = +\infty$	$\alpha = -\infty,\ \beta = 5$	5
J	MIN	$\alpha = 5,\ \beta = +\infty$	$\alpha = 5,\ \beta = 2$	≤ 2
E	MAX	$\alpha = -\infty,\ \beta = 5$	$\alpha = 9,\ \beta = 5$	≥ 9
K	MIN	$\alpha = -\infty,\ \beta = 5$	$\alpha = -\infty,\ \beta = 5$	9
F	MAX	$\alpha = -\infty,\ \beta = 5$	$\alpha = 10,\ \beta = 5$	≥ 10
M	MIN	$\alpha = -\infty,\ \beta = 5$	$\alpha = -\infty,\ \beta = 5$	10
C	MIN	$\alpha = 5,\ \beta = +\infty$	$\alpha = 5,\ \beta = 8$	8
G	MAX	$\alpha = 5,\ \beta = +\infty$	$\alpha = 8,\ \beta = +\infty$	8
O	MIN	$\alpha = 5,\ \beta = +\infty$	$\alpha = 5,\ \beta = 3$	≤ 3
P	MIN	$\alpha = 5,\ \beta = +\infty$	$\alpha = 5,\ \beta = 8$	8
H	MAX	$\alpha = 5,\ \beta = 8$	$\alpha = 24,\ \beta = 8$	24
Q	MIN	$\alpha = 5,\ \beta = 8$	$\alpha = 5,\ \beta = 1$	≤ 1
R	MIN	$\alpha = 5,\ \beta = 8$	$\alpha = 5,\ \beta = 8$	24

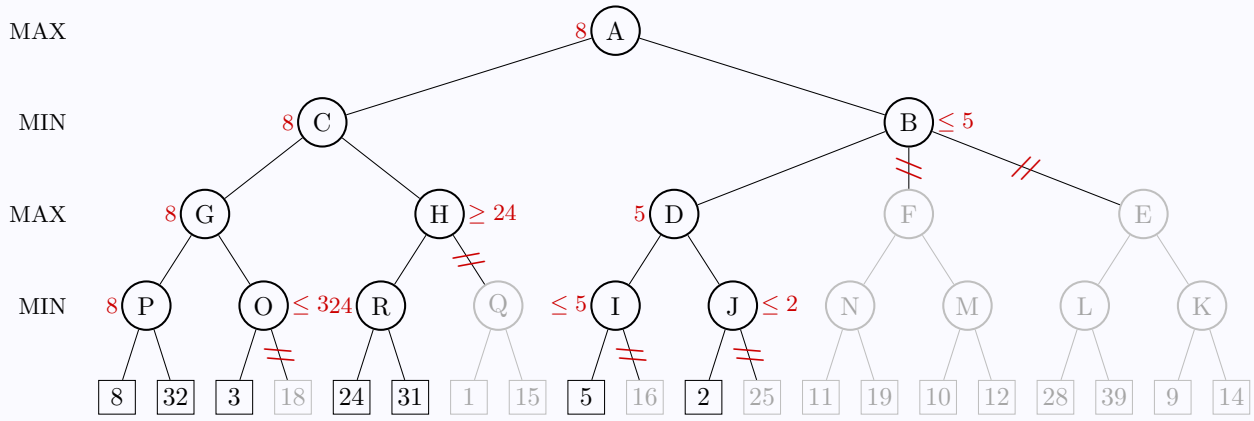
- At MIN node **J**, after leaf #3: $v = 2 \leq \alpha = 5 \Rightarrow$ prune leaf #4. (MAX at D already has an alternative worth $\alpha = 5$, so it will never let play reach J.)
- At MAX node **E**, after child K: $v = 9 \geq \beta = 5 \Rightarrow$ prune L (leaves #7–#8). (MIN at B already has an alternative worth $\beta = 5$, so it will never let play reach E.)

- At MAX node **F**, after child M: $v = 10 \geq \beta = 5 \Rightarrow$ prune N (leaves #11–#12). (MIN at B already has an alternative worth $\beta = 5$, so it will never let play reach F.)
- At MIN node **O**, after leaf #13: $v = 3 \leq \alpha = 5 \Rightarrow$ prune leaf #14. (MAX at A already has an alternative worth $\alpha = 5$, so it will never let play reach O.) **Deep cut-off**: the bound comes from A, 3 levels above O.
- At MIN node **Q**, after leaf #17: $v = 1 \leq \alpha = 5 \Rightarrow$ prune leaf #18. (MAX at A already has an alternative worth $\alpha = 5$, so it will never let play reach Q.) **Deep cut-off**: the bound comes from A, 3 levels above Q.

Leaves evaluated: 13 of 20.

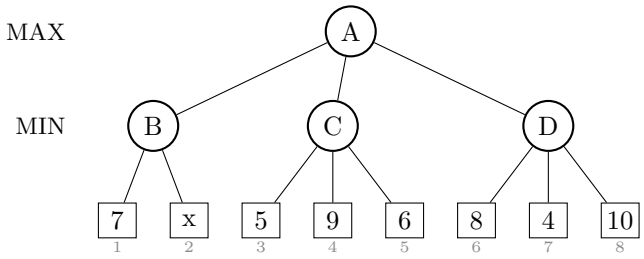
(c) Using the minimax values from (a), and without performing any alpha-beta calculation, rotate the children of each node at every level so that alpha-beta prunes as much as possible. Draw the new tree (keep the node letters).

Best child first for the player to move (MAX: largest value first, MIN: smallest first):



With this ordering alpha-beta evaluates only **7** leaves (instead of 13).

Question 5. The game tree below contains an unknown integer leaf value x. Alpha-beta visits children left to right.



(a) For which values of x are leaves #4 and #5 pruned?

After B, $\alpha = \min(7, x)$. C sees leaf #3 = 5, so it is cut off iff $5 \leq \min(7, x)$, i.e. $\mathbf{x \geq 5}$.

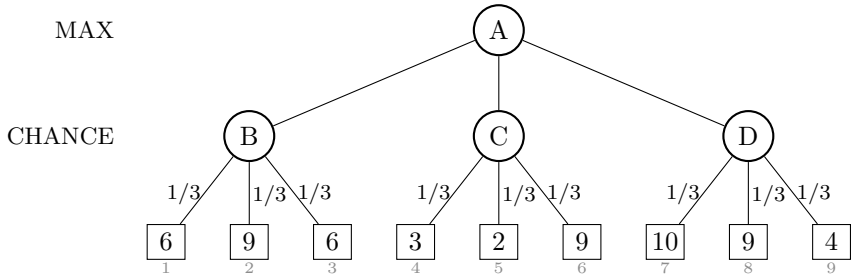
(b) Is leaf #8 ever evaluated?

Never: $\alpha \geq 5$ when D is entered; after 8 and then 4, $v_D = 4 \leq \alpha$, so leaf #8 (10) is pruned.

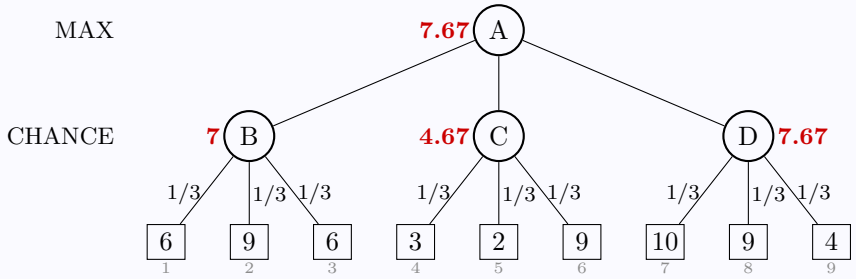
(c) Give the root value as a function of x and say for which x MAX chooses B.

Root $= \max(\min(7, x), 5, 4)$: 5 if $x \le 5$; x if $5 \le x \le 7$; 7 if $x \ge 7$. MAX chooses B iff $x \ge 5$ (tie at $x = 5$ keeps the left-most child). Checked for $x = 0, \ldots, 12$.

Question 6. Consider the expectimax tree below; each chance node has three equally likely outcomes.



(a) Calculate the expectimax value of every node.

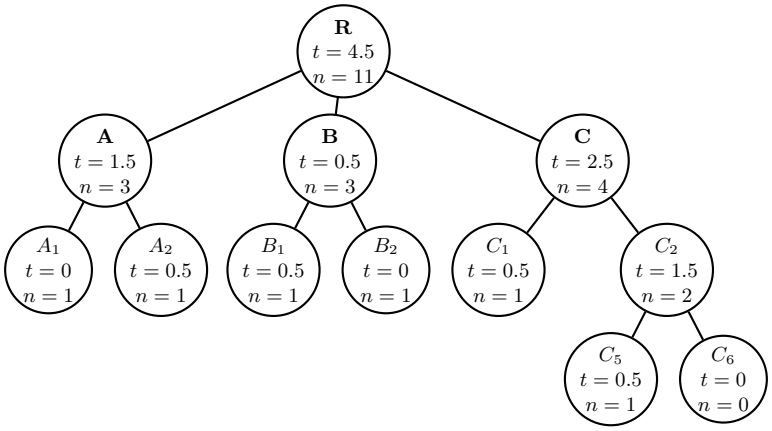


B $= 7$, C $= 14/3 \approx 4.67$, D $= 23/3 \approx 7.67$; MAX chooses D.

(b) Now assume every leaf value is at most 10 (and at least 0). Evaluating leaves left to right, identify the branches that can be pruned and explain each.

After B, $\alpha = 7$. At C, after leaves 3 and 2 the best possible value is $(3 + 2 + 10)/3 = 5 \le 7 \Rightarrow$ **prune leaf #6**. At D, after 10 the bound is 10, after 9 it is 9.67, both > 7, so leaf #9 must be seen. Only leaf #6 is pruned. Without an upper bound on the values no pruning is possible.

Question 7. Rollouts return 1 (win), 0.5 (draw) or 0 (loss) for the root player. Use UCB1 $= \bar{V}_i + C\sqrt{\ln N / n_i}$ (unvisited $= \infty$, ties to the left; a never-visited leaf is rolled out directly, otherwise all its children are added and the rollout starts at the first; $X_1 \to X_3, X_4$ and $X_2 \to X_5, X_6$ for $X \in \{A, B, C\}$). The current tree is:



(a) With $C = \sqrt{2}$, calculate UCB1 of A, B, C. Which child is selected? For which values of C would the root select C instead of A?

A: $\frac{1.5}{3} + \sqrt{2}\sqrt{\frac{\ln 11}{3}} = 1.764$; B: $\frac{0.5}{3} + \sqrt{2}\sqrt{\frac{\ln 11}{3}} = 1.431$; C: $\frac{2.5}{4} + \sqrt{2}\sqrt{\frac{\ln 11}{4}} = 1.720 \Rightarrow$ **A**. C is preferred iff $0.625 + c\sqrt{\ln 11/4} > 0.5 + c\sqrt{\ln 11/3}$, i.e. $c < \frac{0.625-0.5}{0.8940-0.7743} \approx$ **1.04**: a small constant favours exploitation (best average), a large one exploration of the less-visited A.

(b) Perform three iterations whose rollouts return 0, 0.5, 0 (in order); update the statistics and draw the final tree.

Iteration 1.

- *Selection* at R ($N = 11$): A: $\frac{1.5}{3} + \sqrt{2}\sqrt{\frac{\ln 11}{3}} = 1.764$; B: $\frac{0.5}{3} + \sqrt{2}\sqrt{\frac{\ln 11}{3}} = 1.431$; C: $\frac{2.5}{4} + \sqrt{2}\sqrt{\frac{\ln 11}{4}} = 1.720 \Rightarrow$ go to **A**.
- *Selection* at A ($N = 3$): A_1: $\frac{0}{1} + \sqrt{2}\sqrt{\frac{\ln 3}{1}} = 1.482$; A_2: $\frac{0.5}{1} + \sqrt{2}\sqrt{\frac{\ln 3}{1}} = 1.982 \Rightarrow$ go to A_2.
- *Expansion*: A_2 was visited before ($n > 0$) and has no children $\Rightarrow$ add A_5, A_6; roll out from the first one, A_5.
- *Simulation*: rollout from A_5 returns 0.
- *Backpropagation*: add 0 to t and 1 to n for R $\rightarrow$ A $\rightarrow A_2 \rightarrow A_5$.

Statistics (t/n) after iteration 1: R: 4.5/12, A: 1.5/4, A_1: 0/1, A_2: 0.5/2, A_5: 0/1, A_6: 0/0, B: 0.5/3, B_1: 0.5/1, B_2: 0/1, C: 2.5/4, C_1: 0.5/1, C_2: 1.5/2, C_5: 0.5/1, C_6: 0/0.

Iteration 2.

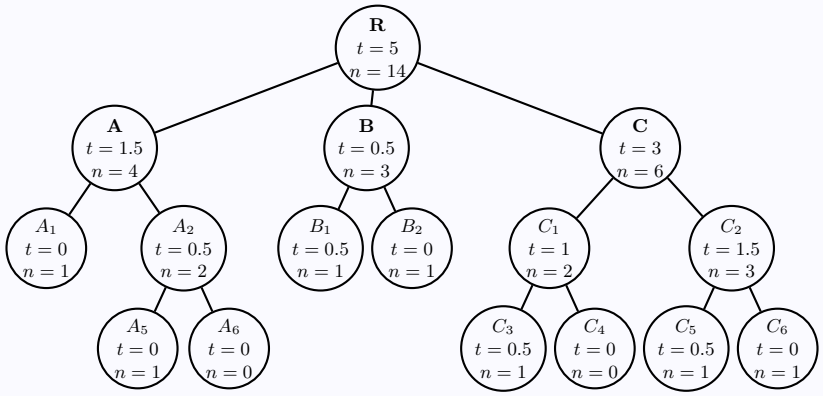
- *Selection* at R ($N = 12$): A: $\frac{1.5}{4} + \sqrt{2}\sqrt{\frac{\ln 12}{4}} = 1.490$; B: $\frac{0.5}{3} + \sqrt{2}\sqrt{\frac{\ln 12}{3}} = 1.454$; C: $\frac{2.5}{4} + \sqrt{2}\sqrt{\frac{\ln 12}{4}} = 1.740 \Rightarrow$ go to **C**.
- *Selection* at C ($N = 4$): C_1: $\frac{0.5}{1} + \sqrt{2}\sqrt{\frac{\ln 4}{1}} = 2.165$; C_2: $\frac{1.5}{2} + \sqrt{2}\sqrt{\frac{\ln 4}{2}} = 1.927 \Rightarrow$ go to C_1.
- *Expansion*: C_1 was visited before ($n > 0$) and has no children $\Rightarrow$ add C_3, C_4; roll out from the first one, C_3.
- *Simulation*: rollout from C_3 returns 0.5.
- *Backpropagation*: add 0.5 to t and 1 to n for R $\rightarrow$ C $\rightarrow C_1 \rightarrow C_3$.

Statistics (t/n) after iteration 2: R: 5/13, A: 1.5/4, A_1: 0/1, A_2: 0.5/2, A_5: 0/1, A_6: 0/0, B: 0.5/3, B_1: 0.5/1, B_2: 0/1, C: 3/5, C_1: 1/2, C_3: 0.5/1, C_4: 0/0, C_2: 1.5/2, C_5: 0.5/1, C_6: 0/0.

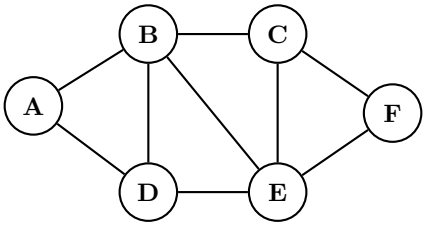
Iteration 3.

- *Selection* at R ($N = 13$): A: $\frac{1.5}{4}+\sqrt{2}\sqrt{\frac{\ln 13}{4}} = 1.507$; B: $\frac{0.5}{3}+\sqrt{2}\sqrt{\frac{\ln 13}{3}} = 1.474$; C: $\frac{3}{5}+\sqrt{2}\sqrt{\frac{\ln 13}{5}} = 1.613 \Rightarrow$ go to **C**.
- *Selection* at C ($N = 5$): C_1: $\frac{1}{2} + \sqrt{2}\sqrt{\frac{\ln 5}{2}} = 1.769$; C_2: $\frac{1.5}{2} + \sqrt{2}\sqrt{\frac{\ln 5}{2}} = 2.019 \Rightarrow$ go to C_2.
- *Selection* at C_2 ($N = 2$): C_5: $\frac{0.5}{1} + \sqrt{2}\sqrt{\frac{\ln 2}{1}} = 1.677$; C_6: ∞ (unvisited) $\Rightarrow$ go to C_6.
- *Expansion*: none — C_6 has $n = 0$, so we roll out directly from it.
- *Simulation*: rollout from C_6 returns 0.
- *Backpropagation*: add 0 to t and 1 to n for R $\rightarrow$ C $\rightarrow C_2 \rightarrow C_6$.

Statistics (t/n) after iteration 3: R: 5/14, A: 1.5/4, A_1: 0/1, A_2: 0.5/2, A_5: 0/1, A_6: 0/0, B: 0.5/3, B_1: 0.5/1, B_2: 0/1, C: 3/6, C_1: 1/2, C_3: 0.5/1, C_4: 0/0, C_2: 1.5/3, C_5: 0.5/1, C_6: 0/1.
Final tree:



Question 8. This problem is about map colouring. Each region must be coloured Red (R), Green (G) or Blue (B), and neighbouring regions must have different colours. The constraint graph is shown below. The following questions are mutually independent.



(a) Cross out all values that would be eliminated by *forward checking* after variable B has been assigned as shown.

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	R G B	**R**	R G B	R G B	R G B	R G B

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	~~R~~ G B	**R** ~~G B~~	~~R~~ G B	~~R~~ G B	~~R~~ G B	R G B

Only the neighbours of B (A, C, D, E) lose R.

(b) A and B have been assigned as shown, but no constraint propagation has been done yet. **Cross out** all values that would be eliminated by *arc consistency* (AC-3).

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	**R**	**G**	R G B	R G B	R G B	R G B

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	**R** ~~G B~~	~~R~~ **G** ~~B~~	~~R G~~ B	~~R G~~ B	R ~~G B~~	~~R~~ G ~~B~~

Propagation continues through the graph (unlike forward checking): every arc $X_i \to X_j$ is revised until no domain changes. Here arc consistency even leaves a single value for every variable.

(c) Variable A is already assigned and constraint propagation has been done. **Circle** all unassigned variables that might be selected by the *Minimum-Remaining-Values* (MRV) heuristic.

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	**R**	G B	R G B	G B	R G B	R G B

Remaining values: B: 2, C: 3, D: 2, E: 3, F: 3 ⇒ MRV may select **B, D**.

(d) Same situation. **Circle** all unassigned variables that might be selected by the *Degree Heuristic*.

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	**R**	G B	R G B	G B	R G B	R G B

Degree = number of constraints with other *unassigned* variables: B: 3, C: 3, D: 2, E: 4, F: 2 ⇒ the degree heuristic may select **E**.

(e) Assume no variable has been assigned yet. Solve the CSP using backtracking with *forward checking*, choosing variables with MRV, then the degree heuristic (DH), and finally alphabetical order; try values in the order R, G, B. For every step give the MRV value of every region not yet coloured, and the DH values of the regions tied on the minimum MRV.

Step 1

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	3	3	3	3	3	3
DH	2	4	3	3	4	2

Assign to the variable **B** with the value **R** (tie on MRV and DH, broken alphabetically). Domains before this step: A=RGB, B=RGB, C=RGB, D=RGB, E=RGB, F=RGB.

Step 2

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	2	–	2	2	2	3
DH	1		2	2	3	

Assign to the variable **E** with the value **G** (tie on MRV broken by DH). Domains before this step: A=GB, B=R, C=GB, D=GB, E=GB, F=RGB.

Step 3

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	2	–	1	1	–	2
DH			1	1		

Assign to the variable **C** with the value **B** (tie on MRV and DH, broken alphabetically). Domains before this step: A=GB, B=R, C=B, D=B, E=G, F=RB.

Step 4

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	2	–	–	1	–	1
DH				1		0

Assign to the variable **D** with the value **B** (tie on MRV broken by DH). Domains before this step: A=GB, B=R, C=B, D=B, E=G, F=R.

Step 5

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	1	–	–	–	–	1
DH	0					0

Assign to the variable **A** with the value **G** (tie on MRV and DH, broken alphabetically). Domains before this step: A=G, B=R, C=B, D=B, E=G, F=R.
Step 6

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	–	–	–	–	–	1
DH						

Assign to the variable **F** with the value **R** (unique minimum MRV). Domains before this step: A=G, B=R, C=B, D=B, E=G, F=R.
Solution: A=G, B=R, C=B, D=B, E=G, F=R. No backtracking is needed.

Question 9. Cryptarithmetic:
$$\begin{array}{r} T\,O \\ +\,G\,O \\ \hline O\,U\,T \end{array}$$
— distinct letters are distinct digits and leading digits are non-zero.

(a) Formulate the CSP: variables (including carries), domains and constraints.

Variables T, O, G, U, carries C_1, C_2; domains $\{0..9\}$, carries $\{0, 1\}$. Constraints: Alldiff(T, O, G, U); $O + O = T + 10C_1$; $C_1 + T + G = U + 10C_2$; $C_2 = O$; $T, G, O \neq 0$.

(b) Solve it by constraint propagation, explaining each deduction.

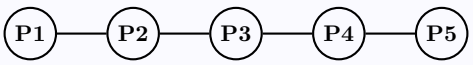
$O = C_2 \in \{0, 1\}$, $O \neq 0 \Rightarrow O = 1$. $2 = T + 10C_1 \Rightarrow T = 2, C_1 = 0$. $2 + G = U + 10 \Rightarrow G = U + 8$, so $(G, U) \in \{(8, 0), (9, 1)\}$; $U = 1$ clashes with $O \Rightarrow G = 8, U = 0$: **21 + 81 = 102**.

Question 10. *Lab scheduling.* Five lab sessions must each be run by one TA, and a TA cannot run two overlapping sessions: P1 8:00–9:30, P2 9:00–10:30, P3 9:30–11:00, P4 10:30–12:00, P5 11:30–13:00 (sessions that only touch do not overlap). TA **H** can run P1, P2, P4, P5; **L** can run P2, P3; **M** can run P1, P3, P4.

(a) Formulate the CSP (one variable per session; apply the availability constraints to the domains) and draw the constraint graph.

Variables	**P1**	**P2**	**P3**	**P4**	**P5**
Domains	H, M	H, L	L, M	H, M	H

Binary constraints: $P1 \neq P2$, $P2 \neq P3$, $P3 \neq P4$, $P4 \neq P5$



(b) Run AC-3 with the initial queue $P1 \to P2$, $P2 \to P1$, $P2 \to P3$, $P3 \to P2$, $P3 \to P4$, $P4 \to P3$, $P4 \to P5$, $P5 \to P4$. Cross out the values it eliminates from your domains in (a), and give the solution.

#	Arc popped	Removed from tail	New domain	Arcs appended to queue
1	$P1 \to P2$	–		–
2	$P2 \to P1$	–		–
3	$P2 \to P3$	–		–
4	$P3 \to P2$	–		–
5	$P3 \to P4$	–		–
6	$P4 \to P3$	–		–
7	$P4 \to P5$	H	$D_{P4} = \{M\}$	$P3 \to P4$
8	$P5 \to P4$	–		–
9	$P3 \to P4$	M	$D_{P3} = \{L\}$	$P2 \to P3$
10	$P2 \to P3$	L	$D_{P2} = \{H\}$	$P1 \to P2$
11	$P1 \to P2$	H	$D_{P1} = \{M\}$	–

Variables	**P1**	**P2**	**P3**	**P4**	**P5**
Domains	~~H~~ M	H ~~L~~	L ~~M~~	~~H~~ M	H

Every domain becomes a singleton: P1=M, P2=H, P3=L, P4=M, P5=H.

Question 11. Answer the following questions briefly.

(a) Match each field with its contribution to AI: (1) Philosophy, (2) Mathematics, (3) Economics, (4) Neuroscience, (5) Psychology, (6) Control theory. (a) neurons as information-processing units; (b) utility and decision theory; (c) logic, reasoning, mind as a physical system; (d) maximising an objective function over time; (e) formal proof, algorithms, (in)tractability, probability; (f) how people perceive, process information and represent knowledge.

(1)c, (2)e, (3)b, (4)a, (5)f, (6)d.

(b) Order chronologically and explain the AI winter: Deep Blue beats Kasparov; Dartmouth meeting; McCulloch & Pitts; AlphaGo beats Lee Sedol; Turing's "Computing Machinery and Intelligence"; the expert-systems boom.

1943 McCulloch & Pitts, 1950 Turing, 1956 Dartmouth, 1980–88 expert-systems boom, 1997 Deep Blue, 2016 AlphaGo. The AI winter (1988–93) was the bust after the expert-systems boom, when expectations were not met and funding collapsed.

(c) For the 8-puzzle below compute h_1 (misplaced tiles) and h_2 (total Manhattan distance). Which heuristic dominates the other, and what does dominance imply for A*?

Start:

4	1	2
	8	7
6	3	5

Goal:

	1	2
3	4	5
6	7	8

$h_1 =$ **5** (tiles 3, 4, 5, 7, 8); $h_2 =$ **9** (1:0, 2:0, 3:2, 4:2, 5:1, 6:0, 7:2, 8:2). h_2 dominates h_1: every misplaced tile is at Manhattan distance ≥ 1, so $h_2(n) \geq h_1(n)$ for all n. With both consistent, A* using h_2 never expands more nodes than A* using h_1, since every node surely expanded with h_2 ($f < C^*$) is also surely expanded with h_1.

(d) A CSP has $n = 5$ variables with domain size $d = 3$. How many leaves does a naive search tree (any unassigned variable at each level) have, and how many with backtracking? Which property justifies the difference?

Naive: $n!\,d^n = 120 \cdot 243 = \mathbf{29{,}160}$ leaves for only $d^n = 243$ complete assignments. Assignments are **commutative**, so backtracking fixes one variable per level: $d^n = \mathbf{243}$ leaves.

Question 12. True or false? For each statement, circle **T** or **F** and justify your answer in one or two sentences (an answer without a correct justification earns no credit).

1. Uniform-cost search expands nodes in order of their optimal path cost. **T** **F**

TRUE. By the separation property, the first time a state is expanded its g is optimal (positive costs).

2. A* is optimally efficient among optimal algorithms using the same heuristic. **T** **F**

TRUE. Any such algorithm must expand all nodes with $f < C^*$, otherwise it could miss the optimum.

3. If $h(n) = 0$ for every node, A* behaves exactly like UCS. **T** **F**

TRUE. $f = g + 0 = g$.

4. If h_2 dominates h_1 and both are consistent, A* with h_2 never expands more nodes than A* with h_1 (ignoring ties). **T** **F**

TRUE. $h_1 \leq h_2 < C^* - g$ for every node surely expanded with h_2.

5. Local beam search with k states is the same as k independent random-restart hill climbs run in parallel. **T** **F**

FALSE. The k best successors are chosen from the pooled list, so information passes between the threads.

6. The amount of alpha-beta pruning does not depend on the move ordering. **T** **F**

FALSE. Perfect ordering gives $O(b^{m/2})$, random about $O(b^{3m/4})$, the worst ordering no pruning at all.

7. Against a sub-optimal MIN player, the minimax move always gives MAX the best possible outcome. **T** **F**

FALSE. MAX does at least as well as against an optimal MIN, but a riskier move may do better.

8. MCTS grows its tree symmetrically, level by level. **T** **F**

FALSE. It grows asymmetrically towards the more promising moves.

9. Node consistency concerns binary constraints. **T** **F**

FALSE. It concerns unary constraints; arc consistency handles binary ones.

10. The degree heuristic is mainly used as a tie-breaker for MRV. **T** **F**

TRUE. At the start all domains are full, so MRV ties and degree decides.